



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2213 - LÓGICA PARA CIENCIAS DE LA COMPUTACIÓN
PROFESOR: MIGUEL ROMERO
AYUDANTES: ALEXANDER PINTO Y ALEJANDRA SCHILD

Ayudantía 9

Problema 1. Considere dos teorías Σ_1 y Σ_2 sobre un mismo vocabulario $\mathcal{L}$. Demuestre que $\Sigma_1 \cap \Sigma_2$ es también una teoría. ¿Es siempre cierto que $\Sigma_1 \cup \Sigma_2$ es una teoría?

Solución: Primero demostraremos que $\Sigma_1 \cap \Sigma_2$ es consistente. Supongamos por contradicción que no lo es. Entonces $\Sigma_1 \cap \Sigma_2 \vdash \alpha$, donde α es una contradicción (una oración no satisfacible). Como $\Sigma_1 \cap \Sigma_2 \subseteq \Sigma_1$, esto implica que $\Sigma_1 \vdash \alpha$. Pero entonces Σ_1 sería inconsistente, lo que contradice el hecho de que es una teoría. Esto prueba que $\Sigma_1 \cap \Sigma_2$ es consistente.

Ahora veamos que $\Sigma_1 \cap \Sigma_2$ es cerrado bajo consecuencia lógica. Para ello, sea φ una $\mathcal{L}$ -oración tal que $\Sigma_1 \cap \Sigma_2 \models \varphi$. Como $\Sigma_1 \cap \Sigma_2 \subseteq \Sigma_1$ y $\Sigma_1 \cap \Sigma_2 \subseteq \Sigma_2$, esto implica que $\Sigma_1 \models \varphi$ y que $\Sigma_2 \models \varphi$. Como Σ_1 es cerrado bajo consecuencia lógica, tenemos que $\varphi \in \Sigma_1$, y análogamente tenemos que $\varphi \in \Sigma_2$. Luego $\varphi \in \Sigma_1 \cap \Sigma_2$, como queríamos. Esto prueba que $\Sigma_1 \cap \Sigma_2$ es una teoría.

Ahora veamos un ejemplo para el cual $\Sigma_1 \cup \Sigma_2$ no es una teoría. Sea $\mathcal{L}$ un vocabulario con un símbolo de constante c y un predicado unario P . Notemos que tanto $\{P(c)\}$ como $\{\neg P(c)\}$ son consistentes, por lo que podemos considerar las teorías $\Sigma_1 := \text{Th}(\{P(c)\})$ y $\Sigma_2 := \text{Th}(\{\neg P(c)\})$. Pero entonces $\Sigma_1 \cup \Sigma_2$ no puede ser una teoría porque es inconsistente. $\square$

Problema 2. Sea Σ una teoría completa sobre un vocabulario $\mathcal{L}$, y sea Γ un conjunto satisfacible de $\mathcal{L}$ -oraciones. Demuestre que, si $\Sigma \subseteq \Gamma$, entonces $\Sigma = \Gamma$.

Solución: Supongamos por contradicción que existe una $\mathcal{L}$ -oración $\varphi \in \Gamma \setminus \Sigma$. Como $\varphi \notin \Sigma$ y Σ es completa, tenemos que $\neg\varphi \in \Sigma$. Pero $\Sigma \subseteq \Gamma$, así que $\neg\varphi \in \Gamma$. Entonces Γ contiene tanto a φ como a $\neg\varphi$, por lo que no puede ser satisfacible, absurdo. $\square$

Problema 3. Sean $L_1, L_2 \subseteq \Sigma^*$ lenguajes en NP. Demuestre que $L_1 \cap L_2$ y $L_1 \cup L_2$ también están en NP.

Solución: Como $L_1 \in \text{NP}$, existe un polinomio $p_1: \mathbb{N} \rightarrow \mathbb{N}$ y una máquina de Turing determinista de tiempo polinomial V_1 tal que para toda palabra $w \in \Sigma^*$ se cumple que

$$w \in L_1 \iff \text{existe } x \in \Sigma^* \text{ tal que } |x| \leq p_1(|w|) \text{ y } V_1 \text{ acepta } \langle w, x \rangle.$$

Similarmente, como $L_2 \in \text{NP}$, existe un polinomio $p_2: \mathbb{N} \rightarrow \mathbb{N}$ y una máquina de Turing determinista de tiempo polinomial V_2 tal que para toda palabra $w \in \Sigma^*$ se cumple que

$$w \in L_2 \iff \text{existe } x \in \Sigma^* \text{ tal que } |x| \leq p_2(|w|) \text{ y } V_2 \text{ acepta } \langle w, x \rangle.$$

Como certificados para decir que $w \in L_1 \cap L_2$ usaremos pares $\langle x_1, x_2 \rangle$, donde x_1 es un certificado de que $w \in L_1$ y x_2 es un certificado de que $w \in L_2$. Estos certificados tienen largo $O((p_1 + p_2)(|w|))$ (polinomial).

Lo que debe hacer el verificador al recibir $\langle w, \langle x_1, x_2 \rangle \rangle$ es ejecutar V_1 con entrada $\langle w, x_1 \rangle$ y V_2 con entrada $\langle w, x_2 \rangle$, y aceptar si y solo si ambas ejecuciones son de aceptación. Esto funciona en tiempo polinomial porque tanto V_1 como V_2 lo hacen.

Como certificados para decir que $w \in L_1 \cup L_2$ usaremos pares $\langle i, x \rangle$, donde $i \in \{1, 2\}$ y x es un certificado de que $w \in L_i$. Estos certificados tienen largo $O\left(\max(p_1(|w|), p_2(|w|))\right)$ (polinomial). Lo que debe hacer el verificador al recibir $\langle w, \langle i, x \rangle \rangle$ es ejecutar V_i con entrada $\langle w, x \rangle$ y aceptar si y solo esta ejecución es de aceptación. Esto funciona en tiempo polinomial porque tanto V_1 como V_2 lo hacen. $\square$

Problema 4. Demuestre que el siguiente lenguaje de pares de enteros (dados en binario) está en NP:

$$L := \{ \langle n, k \rangle \mid n > k > 1 \text{ y } n \text{ no tiene un divisor } d \text{ tal que } 1 < d \leq k \}.$$

Para este problema puede asumir que existe un algoritmo de tiempo polinomial para decidir si un entero dado en binario es primo (algoritmo AKS).

Solución: La observación clave es que si n tiene un divisor d tal que $1 < d \leq k$, entonces también tiene un divisor primo p tal que $1 < p \leq k$ (basta elegir un divisor primo de d). Por lo tanto, para decidir si $\langle n, k \rangle \in L$ nos basta con ver la factorización prima de n y verificar que ninguno de los primos es menor o igual a k . Un certificado para que $\langle n, k \rangle \in L$ es una tupla de primos $\langle p_1, p_2, \dots, p_m \rangle$ (no necesariamente distintos) tales que el producto de ellos es n y que $p_i > k$ para todo i . Como $p_i \geq 2$ para cada i , tenemos que $n = p_1 p_2 \dots p_m \geq 2^m$, de lo que se sigue que $m \leq \log(n)$. Además, $p_i \leq n$ para cada i , así que el largo de estos certificados será a lo sumo $(\log(n))^2$, que es polinomial en el largo de la entrada. Podemos verificar que cada uno de los p_i es efectivamente un primo usando el algoritmo AKS. Por último, necesitamos hacer menos que $\log(n)$ multiplicaciones entre números menores n , y cada una de ellas puede hacerse en tiempo $O(\log(n)^2)$ (ejercicio). $\square$

Problema 5. Demuestre que el siguiente lenguaje de pares de enteros (dados en binario) está en P:

$$\text{RELPRIMES} := \{ \langle a, b \rangle \mid a, b \geq 1 \text{ y } a \text{ y } b \text{ no tienen un divisor común mayor a } 1 \}.$$

Solución: Podemos usar el algoritmo de Euclides para calcular el máximo común divisor del par $\langle a, b \rangle$:

```

1: Inicializar  $x := \max(a, b)$ ;
2: inicializar  $y := \min(a, b)$ ;
3: while  $y > 0$  do
4:   reasignar  $x$  con  $x \bmod y$  (el resto de dividir  $x$  en  $y$ );
5:   intercambiar los valores de  $x$  e  $y$ ;
6: end while
7: retornar  $x$ .
```

Omitiremos la demostración de que el algoritmo de Euclides es correcto. Lo que debemos hacer es ejecutar el algoritmo y verificar si el número que retorna es o no igual a 1. Cada línea del pseudocódigo puede hacerse en tiempo polinomial (en particular, queda de ejercicio demostrar que el algoritmo del colegio para dividir dos números acotados superiormente por n funciona en tiempo $O(\log(n)^2)$). Lo que nos falta probar es que el bucle entre las líneas 3 y 6 se ejecuta un número polinomial de veces (en términos de $\log(a) + \log(b)$, el largo de la entrada). Para probar esto, mostraremos que cada ejecución de la línea 4 reduce el valor de x a a lo sumo la mitad, de lo que se seguirá que el número de iteraciones posibles no puede ser mayor que $\log(a) + \log(b)$. Notemos que, por definición del resto de la división, al principio y al final de cada iteración del bucle tendremos que $x \geq y$. Tenemos dos casos:

- El primer caso es que $\frac{x}{2} \geq y$. Entonces $x \bmod y < y \leq \frac{x}{2}$ (la primera desigualdad es porque el resto de una división es menor que el divisor). Por lo tanto, el valor de x disminuyó a menos de la mitad.
- El segundo caso es que $\frac{x}{2} < y$. Entonces $x \bmod y = x - y < \frac{x}{2}$ (donde la igualdad se tiene porque $x - 2y < 0$ y $x - 0y \geq y$, así que $x - y$ es el resto). En este caso igualmente ocurre lo que queríamos.

$\square$